

Agents SDK Examples Study Order

- Source tree: OpenAI Agents SDK examples

Study Order Notes

- OpenAI does not provide an SDK examples roadmap
- May of the examples have no comments/documentation
- The script OpenAI provides to run the examples, simply goes through them in alphabetical order by folder and file name

How I Created This Document

- Tools like Codex, Claude Code, ..., can understand and explain code bases
- Asked Codex to study all examples and recommend order based on difficulty and features
- Also asked for a brief description of each
- Verified with Claude Code
- Re-verified against the repository's `examples/` tree on September 16, 2026 (commit `5f9899d`)

Codex-Recommended Folder Order

- `basic`
- `tools`
- `agent_patterns`
- `handoffs` / `customer_service`
- `memory`
- `hosted_mcp` / `mcp`
- `research_bot`
- `financial_research_agent`
- `model_providers` / `reasoning_content`
- `voice` / `realtime`
- `sandbox`

`examples/` (top-level files)

Read these as needed; they support the examples rather than teach a feature.

1. `README.md`
   - How OpenAI runs the whole suite with `make examples-run`, filters, and per-example logs.
2. `run_examples.py`
   - The runner that executes every example in alphabetical folder/file order (the "no roadmap" order this notebook replaces).
3. `auto_mode.py`
   - Helpers that supply canned inputs and approvals when `EXAMPLES_INTERACTIVE_MODE=auto`, so interactive examples can run unattended; imported by 33 examples.
4. `web_search_utils.py`
   - Extracts URL citations from web-search results; used by `tools/web_search_filters.py` and the financial research agent.

`examples/basic/`

1. `hello_world.py` (yes)
   - Introduces the smallest runnable Agents SDK flow: define an `Agent`, call `Runner`, and inspect the final answer.
2. `hello_world_jupyter.ipynb` (yes)
   - Notebook version of the first agent run. Readers see how the same `Agent` plus `Runner` pattern works in an interactive teaching environment.
3. `hello_world_gpt_5.py` (yes)
   - Adds explicit GPT-5 model and reasoning settings to the basic agent. Readers see how model choice and `ModelSettings` affect an otherwise simple run.

4. `dynamic_system_prompt.py` (maybe)
    - Shows runtime-generated instructions from context, so one agent can adapt behavior without separate definitions.
5. `tools.py` (yes)
    - Introduces `@function_tool` with a callable the model can invoke. Readers learn the first step from plain text generation toward agents that use application code.
6. `usage_tracking.py` (maybe)
    - Shows where token and request usage are reported after an agent run.
7. `stream_text.py` (yes)
    - Demonstrates streaming text deltas for responsive terminal or notebook output.
8. `stream_items.py` (maybe)
    - Demonstrates higher-level streamed run items such as messages, tool calls, and events.
9. `stream_function_call_args.py` (advanced)
    - Shows how to inspect function-call arguments while the model is still producing them.
10. `previous_response_id.py`
    - Continues a conversation with a prior Responses API response ID instead of resending all history.
11. `local_image.py`
    - Passes a local image as base64 input for multimodal image understanding.
12. `remote_image.py`
    - Passes a hosted image URL as input for a simple vision workflow.
13. `local_file.py`
    - Passes local file content directly into an agent run without a vector store.
14. `remote_pdf.py`
    - Analyzes a PDF by URL, demonstrating direct remote-document input.
15. `image_tool_output.py`
    - Returns image content from a tool, showing that tools can produce multimodal outputs.
16. `non_strict_output_type.py`
    - Explores structured output when a schema cannot use strict JSON mode.
17. `tool_guardrails.py`
    - Combines tool use with guardrail checks so tool workflows are constrained by policy or validation.
18. `agent_lifecycle_example.py`
    - Adds hooks for observing agent/run lifecycle events, tool calls, model responses, and usage.
19. `lifecycle_example.py`
    - Adds hooks for observing agent/run lifecycle events, tool calls, model responses, and usage.

20. `prompt_template.py`
    - Uses a platform prompt template with runtime variables.
21. `retry.py`
    - Configures model retry behavior for transient failures.
22. `retry_litellm.py`
    - Applies retry behavior while using a LiteLLM-backed provider.
23. `hello_world_gpt_oss.py`
    - Runs an agent against a local `gpt-oss` model through an OpenAI-compatible client. Readers learn how the SDK abstractions map to local open-weight model serving.
24. `stream_ws.py`
    - Shows a WebSocket-oriented streaming pattern for service or browser integrations.

`examples/tools/`

1. `web_search.py`
    - Uses OpenAI-hosted web search as an agent tool.
2. `web_search_filters.py`
    - Adds web-search filters and source URL inspection.
3. `file_search.py`
    - Indexes or uses a vector store and queries it through `FileSearchTool` for RAG.
4. `code_interpreter.py`
    - Lets the agent run Python in a hosted container and return generated artifacts.
5. `image_generator.py`
    - Uses `ImageGenerationTool` and saves the generated image result.
6. `shell.py`
    - Implements a shell executor for `ShellTool`, including command output and approval handling.
7. `shell_human_in_the_loop.py`
    - Runs shell commands with explicit human approval before sensitive execution.
8. `skills/csv-workbench/SKILL.md` and `skills/csv-workbench/playbook.md`
    - The sample skill the next three examples load: a `SKILL.md` description plus a playbook of shell commands for summarizing CSV files.
9. `local_shell_skill.py`
    - Attaches the skill from a local folder (`ShellToolLocalSkill`) to a `ShellTool` that runs on your machine, reusing `shell.py`'s executor.
10. `container_shell_inline_skill.py`
    - Zips the skill folder and sends it inline (`ShellToolInlineSkill`) to OpenAI's hosted container shell.

11. `container_shell_skill_reference.py`
    - References a skill already uploaded to OpenAI by ID and version (`ShellToolSkillReference`) from the hosted container shell.
12. `tool_search.py`
    - Demonstrates dynamic tool discovery and namespaced tool exposure.
13. `programmatic_tool_calling.py`
    - Lets the model write a program that calls several `@tool(allowed_callers=['programmatic'])` tools concurrently through `ProgrammaticToolCallingTool`, with `tool_choice='programmatic_tool_calling'`.
14. `apply_patch.py`
    - Uses patch application as a tool for code-editing workflows.
15. `computer_use.py`
    - Backs `ComputerTool` with Playwright so the model can inspect and operate a browser.
16. `codex.py`
    - Delegates software-engineering work to the experimental Codex tool and streams Codex events.
17. `codex_same_thread.py`
    - Reuses one Codex thread so coding work carries forward across turns.

`examples/agent_patterns/`

OpenAI's README frames this folder around key agent patterns

- deterministic flows
- routing/handoffs
- agents-as-tools
- judging
- parallelization
- guardrails
- human-in-the-loop patterns

1. `README.md`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
2. `deterministic.py`
    - Breaks work into fixed Python-orchestrated agent steps.
3. `routing.py`
    - Uses a triage/routing agent and handoffs to specialized agents.
4. `agents_as_tools.py`
    - Uses specialized agents as callable tools instead of handing off control.

5. `agents_as_tools_structured.py`
   - Calls specialized agents as tools with structured input contracts.
6. `agents_as_tools_streaming.py`
   - Streams progress from nested agent-as-tool calls.
7. `agents_as_tools_conditional.py`
   - Enables or disables agent tools based on runtime context or access level.
8. `parallelization.py`
   - Runs multiple agents in parallel and selects or synthesizes the best result.
9. `llm_as_a_judge.py`
   - Uses one model call to judge or improve another model call.
10. `input_guardrails.py`
    - Checks user input before or alongside the main agent run.
11. `output_guardrails.py`
    - Validates final output before returning it to the caller.
12. `streaming_guardrails.py`
    - Checks streamed output during generation so bad output can be stopped early.
13. `forcing_tool_use.py`
    - Forces tool use and compares ways to turn tool results into final output.
14. `human_in_the_loop.py`
    - Serializes an interrupted run, collects approval, and resumes execution.
15. `human_in_the_loop_custom_rejection.py`
    - Customizes how rejected tool approvals are reported back into the run.
16. `human_in_the_loop_stream.py`
    - Combines streaming output with approval interruptions.
17. `hosted_multi_agent_beta.py`
    - Experimental `OpenAIHostedMultiAgentModel`: the root agent asks OpenAI to spawn concurrent subagents server-side, each calling a local `@tool`; shows both streamed and non-streamed runs and how to identify which hosted agent made a call.

`examples/handoffs/`

1. `message_filter.py`
   - Filters what conversation context is passed during a handoff.
2. `message_filter_streaming.py`
   - Filters handoff context in a streamed run.

`examples/customer_service/`

1. `main.py`
   - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.

Study this after `agent_patterns/routing.py` and `handoffs/message_filter.py`; it combines context objects, tools, handoffs, recommended handoff prompts, and streamed item inspection.

`examples/memory/`

1. `sqlite_session_example.py`
   - Persists conversation history locally with SQLite.
2. `openai_session_example.py`
   - Uses OpenAI-managed conversation session storage.
3. `file_session.py`
   - Implements a custom file-backed session adapter.
4. `advanced_sqlite_session_example.py`
   - Persists conversation history locally with SQLite.
5. `encrypted_session_example.py`
   - Wraps session storage with encryption and expiration behavior.
6. `memory_session_hitl_example.py`
   - Combines local session memory with human approval.
7. `openai_session_hitl_example.py`
   - Uses OpenAI-managed conversation session storage.
8. `file_hitl_example.py`
   - Combines file-backed memory with human approval.
9. `hitl_session_scenario.py`
   - Exercises approvals, rehydration, and rejection behavior across sessions.
10. `compaction_session_example.py`
    - Automatically compacts long session history while preserving context.
11. `compaction_session_stateless_example.py`
    - Compacts conversation state when responses are not stored server-side.
12. `redis_session_example.py`
    - Uses Redis as scalable session storage.
13. `mongodb_session_example.py`
    - Uses MongoDB as session storage with an async client.
14. `sqlalchemy_session_example.py`
    - Uses SQLAlchemy so session storage can target different relational databases.

15. `dapr_session_example.py`
    - Uses Dapr state management as a portable session backend.

`examples/hosted_mcp/`

1. `simple.py`
    - Shows the minimal connection to a hosted MCP server.
2. `on_approval.py`
    - Adds approval behavior to hosted MCP tool calls.
3. `human_in_the_loop.py`
    - Serializes an interrupted run, collects approval, and resumes execution.
4. `connectors.py`
    - Demonstrates connector-backed hosted MCP tools.

`examples/mcp/`

1. `filesystem_example/README.md`, then `filesystem_example/main.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
2. `git_example/README.md`, then `git_example/main.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
3. `streamablehttp_example/README.md`, then `server.py`, then `main.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
4. `sse_example/README.md`, then `server.py`, then `main.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
5. `sse_remote_example/README.md`, then `main.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
6. `streamable_http_remote_example/README.md`, then `main.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
7. `streamablehttp_custom_client_example/README.md`, then `server.py`, then `main.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
8. `prompt_server/README.md`, then `server.py`, then `main.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.

9. `tool_filter_example/README.md`, then `main.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
10. `get_all_mcp_tools_example/README.md`, then `main.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
11. `manager_example/README.md`, then `mcp_server.py`, `app.py`, `smoke_test.py`
    - Coordinates planner, worker, and writer agents in a multi-agent workflow.

`examples/research_bot/`

OpenAI's README describes flow as planner -> parallel web searches -> writer report

1. `README.md`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
2. `main.py`
    - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.
3. `manager.py`
    - Coordinates planner, worker, and writer agents in a multi-agent workflow.
4. `agents/planner_agent.py`
    - Builds a planning agent that decomposes a request into research tasks.
5. `agents/search_agent.py`
    - Defines a specialist agent for web/retrieval work.
6. `agents/writer_agent.py`
    - Synthesizes gathered findings into a final report.
7. `printer.py`
    - Separates terminal progress/output formatting from agent logic.
8. `sample_outputs/`

`examples/financial_research_agent/`

1. `README.md`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
2. `main.py`
    - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.
3. `manager.py`
    - Coordinates planner, worker, and writer agents in a multi-agent workflow.

4. `agents/planner_agent.py`
   - Builds a planning agent that decomposes a request into research tasks.
5. `agents/search_agent.py`
   - Defines a specialist agent for web/retrieval work.
6. `agents/financials_agent.py`
   - Adds finance-specific data gathering and interpretation.
7. `agents/risk_agent.py`
   - Focuses a specialist agent on downside and risk analysis.
8. `agents/writer_agent.py`
   - Synthesizes gathered findings into a final report.
9. `agents/verifier_agent.py`
   - Adds a verification pass over the generated financial report.
10. `printer.py`
    - Separates terminal progress/output formatting from agent logic.

`examples/model_providers/`

1. `README.md`
   - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
2. `litellm_auto.py`
   - Uses LiteLLM with minimal setup for non-OpenAI model access.
3. `litellm_provider.py`
   - Configures LiteLLM through the SDK provider abstraction.
4. `any_llm_auto.py`
   - Shows a convenience path for another OpenAI-compatible model provider.
5. `any_llm_provider.py`
   - Configures the alternate provider explicitly.
6. `custom_example_agent.py`
   - Shows agent code using the custom provider setup.
7. `custom_example_provider.py`
   - Implements a custom model provider extension point.
8. `custom_example_global.py`
   - Registers custom provider behavior globally.

`examples/reasoning_content/`

1. `runner_example.py`
   - Inspects reasoning-content behavior in a runnable example.

2. `main.py`
    - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.
3. `gpt_oss_stream.py`
    - Runs an agent against a local open-weight `gpt-oss` model through an OpenAI-compatible interface.

`examples/voice/`

1. `static/README.md`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
2. `static/util.py`
    - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.
3. `static/main.py`
    - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.
4. `streamed/README.md`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
5. `streamed/my_workflow.py`
    - Orchestrates tools, policies, data, and agents into an end-to-end workflow.
6. `streamed/main.py`
    - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.

`examples/realtime/`

1. `cli/demo.py`
    - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.
2. `app/README.md`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
3. `app/agent.py`
    - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.
4. `app/server.py`
    - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.

5. `app/static/`
6. `twilio/README.md`
    - Bridges realtime agents to Twilio voice or SIP media flows.
7. `twilio/server.py`
    - Bridges realtime agents to Twilio voice or SIP media flows.
8. `twilio/twilio_handler.py`
    - Bridges realtime agents to Twilio voice or SIP media flows.
9. `twilio_sip/README.md`
    - Bridges realtime agents to Twilio voice or SIP media flows.
10. `twilio_sip/agents.py`
    - Bridges realtime agents to Twilio voice or SIP media flows.
11. `twilio_sip/server.py`
    - Bridges realtime agents to Twilio voice or SIP media flows.

`examples/sandbox/` (Beta, may change significantly)

OpenAI's sandbox README divides this folder into small API examples, cloud backends, tutorials, workflow examples and shared files

1. `README.md`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
2. `unix_local_runner.py`
    - Implements the local Unix sandbox runner.
3. `basic.py`
    - Runs the minimal sandbox API workflow.
4. `sandbox_agent_capabilities.py`
    - Inspects available sandbox capabilities.
5. `unix_local_pty.py`
    - Shows interactive local sandbox execution through a pseudo-terminal.
6. `sandbox_agent_with_tools.py`
    - Combines sandboxed execution with tool-using agents.
7. `sandbox_agents_as_tools.py`
    - Uses specialized agents as callable tools instead of handing off control.
8. `handoffs.py`
    - Uses handoffs between sandbox-backed agents.
9. `memory.py`
    - Adds state to a sandboxed agent workflow.

10. `memory_multi_agent_multiturn.py`
    - Combines sandbox memory, multiple agents, and multi-turn state.
11. `shared_session_workdirs.py`
    - Runs two trusted agents in separate working directories of one live sandbox, so Shell, `view_image` and `apply_patch` resolve relative paths per run; not a security boundary.
12. `sandbox_agent_with_remote_snapshot.py`
    - Starts sandbox work from a reusable remote environment snapshot.
13. `memory_s3.py`
    - Persists sandbox memory with S3.
14. `tax_prep.py`
    - Builds an end-to-end sandboxed domain workflow around documents and calculations.

`examples/sandbox/docs/`

The runnable example behind the SDK's sandbox-agents documentation: a tiny repo, one lazy-loaded skill, and a self-verifying coding task.

1. `repo/README.md`, then `repo/task.md`
    - The fix the agent must make: a shell script prints a debit label instead of a credit label and keeps the sign.
2. `skills/credit-note-fixer/SKILL.md`
    - The skill that tells the agent how to inspect the repo, apply the smallest patch, and rerun the exact test command.
3. `coding_task.py`
    - Gives the model the repo plus the skill, then verifies that the agent edited the repo and ran the targeted test.

`examples/sandbox/misc/`

Support code shared by the sandbox examples above; read each when the example that imports it comes up.

1. `example_support.py`
    - Builds in-memory manifests and pretty-prints tool calls; imported by most of the small API examples.
2. `workspace_shell.py`
    - A `Capability` that exposes one `ShellTool` for inspecting the active sandbox workspace.
3. `workspace_apply_patch.py`
    - A `Capability` that exposes `ApplyPatchTool` against the sandbox filesystem.
4. `reference_policy_mcp_server.py`
    - A tiny MCP v2 server with one policy-lookup tool, started by `sandbox_agent_with_tools.py`.

`examples/sandbox/tutorials/`

1. `misc.py`
   - Shared helper code for the tutorials.
2. `Dockerfile`
   - The `sandbox-tutorials` image the tutorials run in (`docker build -t sandbox-tutorials:latest -f examples/sandbox/tutorials/Dockerfile .`).
3. `data/dataroom/setup.py`
   - Creates tutorial data or environment fixtures.
4. `sandbox_resume/README.md`, then `main.py`
   - Edits a workspace app and reuses a sandbox snapshot.
5. `dataroom_qa/README.md`, then `main.py`
   - Answers questions over a mounted dataroom with source-backed responses.
6. `dataroom_metric_extract/README.md`, then `schemas.py`, `main.py`, `evals.py`
   - Extracts structured financial metrics to CSV/JSONL and evaluates the results.
7. `repo_code_review/README.md`, then `main.py`, `evals.py`
   - Reviews a sample repo and writes finding, report, and patch artifacts.
8. `vision_website_clone/README.md`, then `skills/playwright/SKILL.md`, then `main.py`
   - Uses vision and a browser-review loop to clone a reference static website.

`examples/sandbox/healthcare_support/`

1. `README.md`
   - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
2. `models.py`
   - Defines domain data models used by the workflow.
3. `data.py`
   - Loads sample domain data for the workflow.
4. `tools.py`
   - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.
5. `support_agents.py`
   - Defines specialized healthcare support agents.
6. `workflow.py`
   - Orchestrates tools, policies, data, and agents into an end-to-end workflow.
7. `main.py`
   - Shows a focused SDK feature in runnable source form. Readers should study it for the setup, agent configuration, run call, and result-handling pattern.

8. `skills/prior-auth-packet-builder/SKILL.md`
9. `policies/`
10. `data/scenarios/`

`examples/sandbox/extensions/`

1. `README.md`
   - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
2. `e2b_runner.py`
   - Implements a sandbox runner for the E2B backend.
3. `modal_runner.py`
   - Implements a sandbox runner for Modal.
4. `cloudflare_runner.py`
   - Implements a sandbox runner for Cloudflare.
5. `blaxel_runner.py`
   - Implements a sandbox runner for Blaxel.
6. `vercel_runner.py`
   - Implements a sandbox runner for Vercel.
7. `daytona/daytona_runner.py`
   - Implements a sandbox runner for Daytona.
8. `daytona/usaspending_text2sql/README.md`, then `schema/`, `sql_capability.py`, `setup_db.py`, `agent.py`
   - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.
9. `runloop/capabilities.py`, then `runloop/runner.py`
   - Defines provider capabilities exposed by a sandbox extension.
10. `temporal/README.md`, then `local_hello_workflow.py`, `_worker_setup.py`, `temporal_sandbox_agent.py`, `temporal_session_manager.py`, `temporal_sandbox_tui.py`
    - Provides setup notes and conceptual framing for the examples in that folder before readers inspect the code.

`examples/sandbox/docker/`

1. `Dockerfile.mount`
2. `docker_runner.py`
   - Implements a provider runner adapter.
3. `mounts/mount_smoke.py`
   - Smoke-tests mounted filesystem behavior.

4. `mounts/s3_mount_read_write.py`
    - Tests S3 mount reads and writes.
5. `mounts/gcs_mount_read_write.py`
    - Tests Google Cloud Storage mount reads and writes.
6. `mounts/azure_mount_read_write.py`
    - Tests Azure mount reads and writes.

© 2026 by Deitel & Associates, Inc. All Rights Reserved.